极客时间

以终为始

L01

一次可验证的 AI 交付怎样完成？

先造自己的 AI 研发工作台，再用它持续开发 FlowERP

01 课程最终作品
02 本课要解决的问题
03 从需求到验收
04 工作台 V0.1 设计
05 动手实践
06 总结

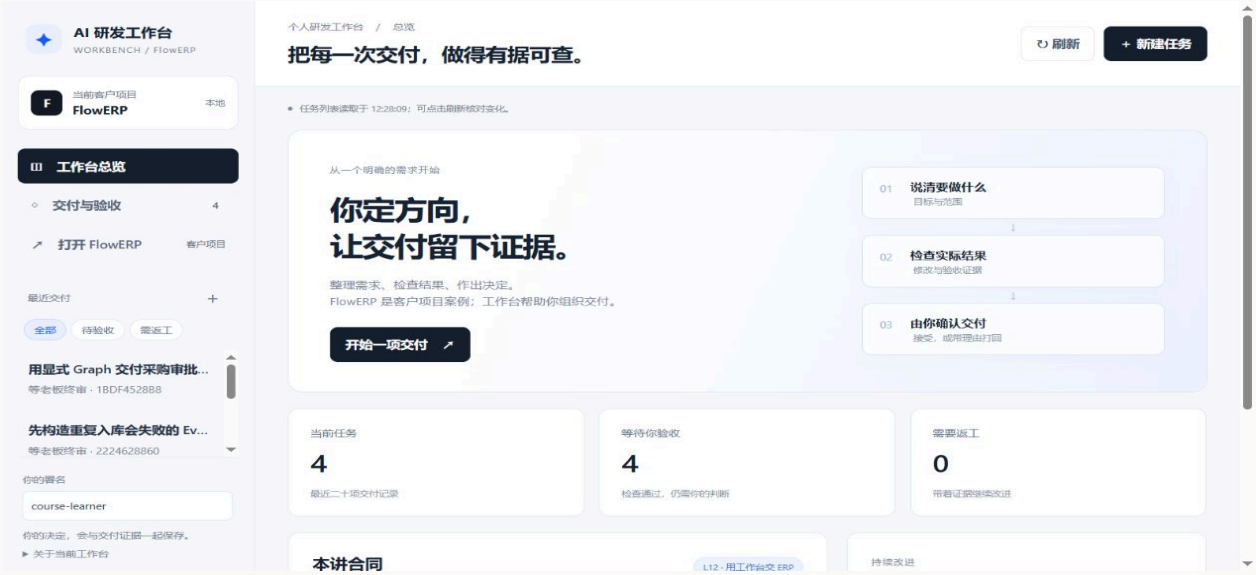
01

先看课程最终作品，再看今天要做什么

先和 Codex 建工作台，再用它开发 FlowERP。
本讲先实现任务和运行记录的保存与查询。

我们要做出一个自己日常使用的 AI 研发工作台

它是我们建设的研发应用：把项目、需求、Codex 的执行、检查结果和验收组织在一起。
需求完整链路：业务问题→产品需求（PRD）→技术方案→开发→测试→线上效果回收



选项目

选代码仓库，读取项目约定

提需求

明确这次做什么、怎样验收

组织开发

调用 Codex，执行约定检查

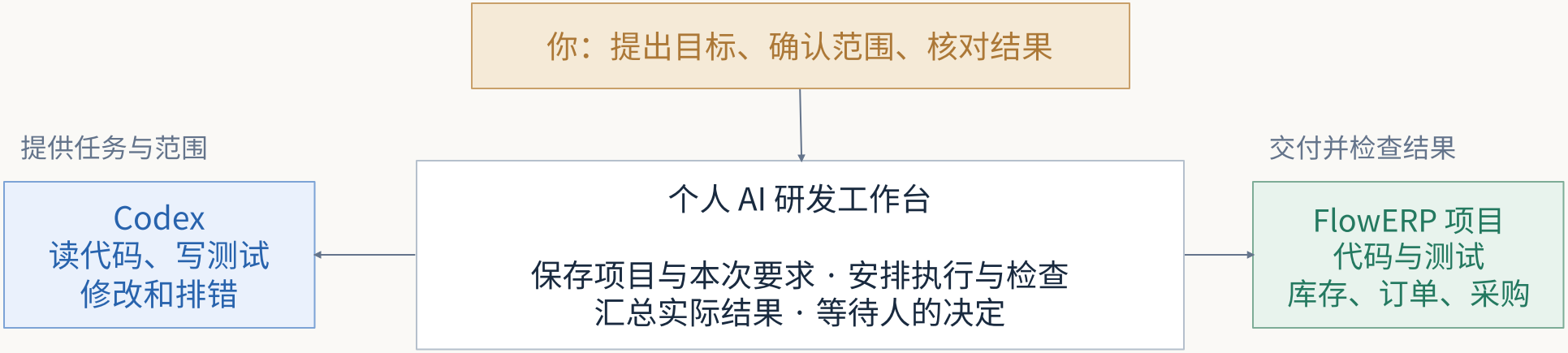
看结果、作决定

查看修改和失败，接受或退回

图为课程参考成品。L01 先做命令行记录功能，L04 补齐执行与检查，后续再完善网页和协作能力。

工作台连接哪些东西?

使用者是开发者。Codex 协助开发，FlowERP 是要持续交付的客户产品。

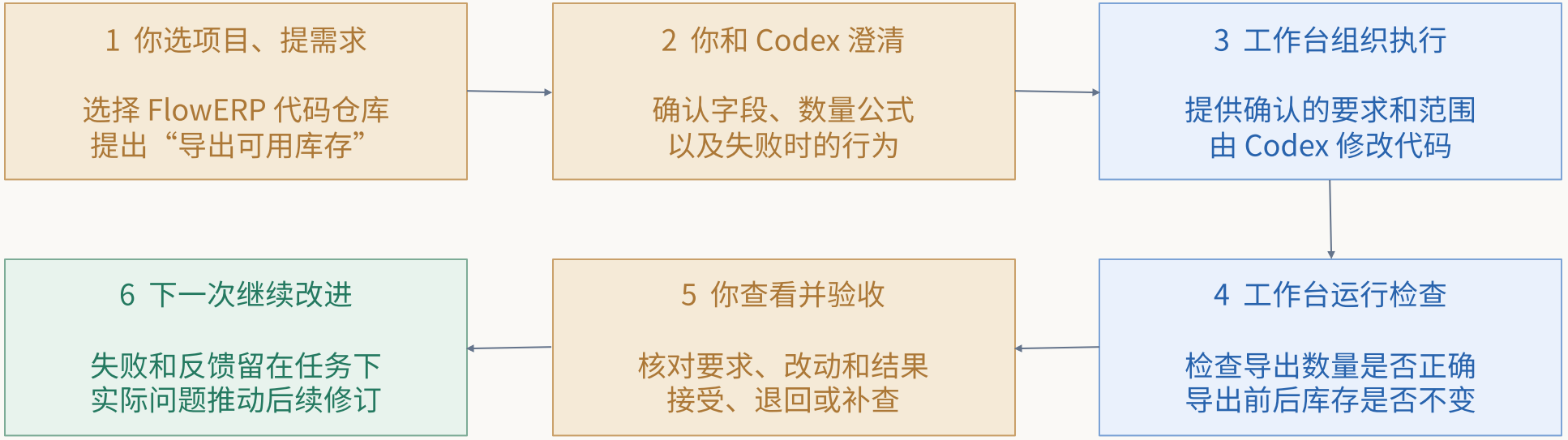


完整工作台的目标：让一次开发从提出需求到检查结果，都能按约定组织并继续跟进。

先用 Codex 开发工作台本身；从 L04 起，再通过工作台组织 Codex 修改 FlowERP。两套应用各有自己的数据。

在工作台里，一项开发任务怎样走完？

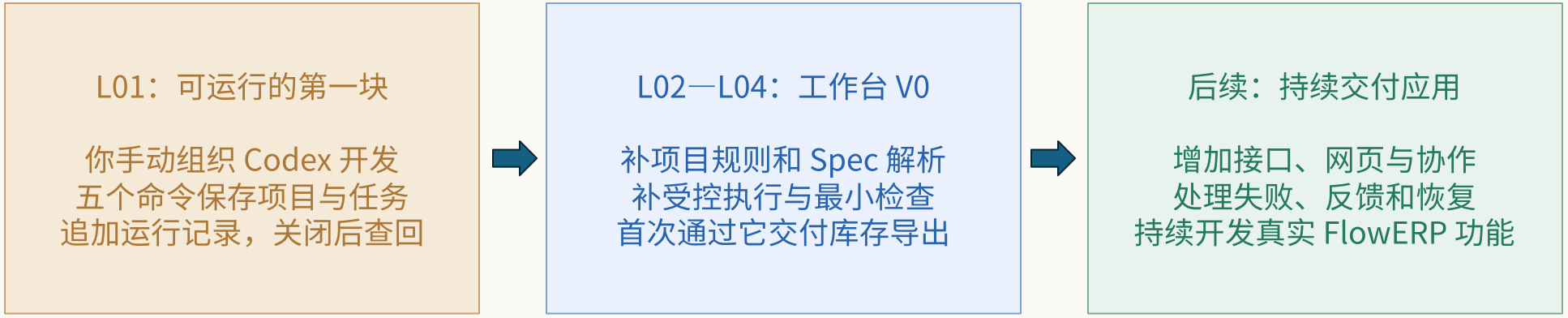
后续交付示例：为 FlowERP 增加库存导出。下面展示目标流程，L01 暂不实现这个业务功能。



检查失败就根据输出修复并重跑；要求变化先由人确认。通过的结果、失败原因与人的决定都回到同一任务。

完整工作台很大，本讲先做哪一块？

第一版就要能运行、能使用。先做记录能力，是为后续执行和检查确定共同的任务与要求。



如果没有保存当时的任务与要求，后续就难以判断：Codex 按什么改、检查的是哪一版、失败后从哪里继续。

01 课程最终作品
02 本课要解决的问题
03 从需求到验收
04 工作台 V0.1 设计
05 动手实践
06 总结

02

已有 Codex，为什么还需要工作台？

哪些记录总要查找？哪些错误反复出现？先找出这些麻烦，再判断做一个工作台是否值得。

为什么值得再建设一套工作台？

直接与 Codex 协作时，你已经在组织开发。重复交接增多后，看看哪些动作值得固定成程序。

每次接手都要找上下文

当时的需求在哪？哪次修改对应哪次测试？

→ 任务编号 + 当时的需求 + 多次运行记录

每次都要核对执行范围

这次允许改什么？要运行哪些检查？

→ 后续把确认的范围与检查入口交给执行流程

失败后很难接着处理

卡在哪一步？哪些结果仍然有效？

→ 保留失败输出、待办事项和人的决定

是否值得做，要比较实际使用效果：接手是否更快、漏项是否减少、失败是否能继续处理，以及维护工作台的成本。

从使用条件推导第一版，别从架构名词开始

本讲的具体条件：个人、本地、任务量不大；先解决隔天接手与检查记录的问题。

具体条件／隐藏要求	本次选择与代价	怎样检查是否合适
隔天还要查，程序会退出	落盘保存；增加持久化实现	退出旧进程，再启动查询
要求会变，但旧任务还要复查	保存当时的需求副本；多占存储	修改原文件后，旧任务仍读旧要求
失败后还会修复，历史不能丢	每次运行单独追加；不能只留最新结果	第一次失败与后续通过都能查回
一人本地先验证，时间有限	先用 CLI 和 SQLite；暂缓网页与远程协作	五个操作能完整走通自身开发任务

这些是本讲的范围选择。多人并发、大量日志或审计要求出现后，应重新比较存储、权限和运行方式。

工作台固定流程，工程判断仍要由你作出

规则、测试和自动化都应来自具体要求。工作台把这些决定用于执行，并保留结果供人检查。

你要判断什么	工作台能怎样帮助	仍然可能漏掉什么
真正的使用目的与隐藏条件	整理问题、来源和确认后的要求	没有被发现或问清的条件
哪些旧代码可复用，改到哪里	提供项目上下文，记录允许范围与改动	复用与耦合是否适合当前项目
怎样判断实际结果可接受	运行约定检查，保留失败与输出	测试没覆盖的业务风险

开发者要解释为什么这样设计，并用实际结果修订。流程全部走完、测试全部变绿，也可能仍有未发现的要求。

01 课程最终作品
02 本课要解决的问题
03 从需求到验收
04 工作台 V0.1 设计
05 动手实践
06 总结

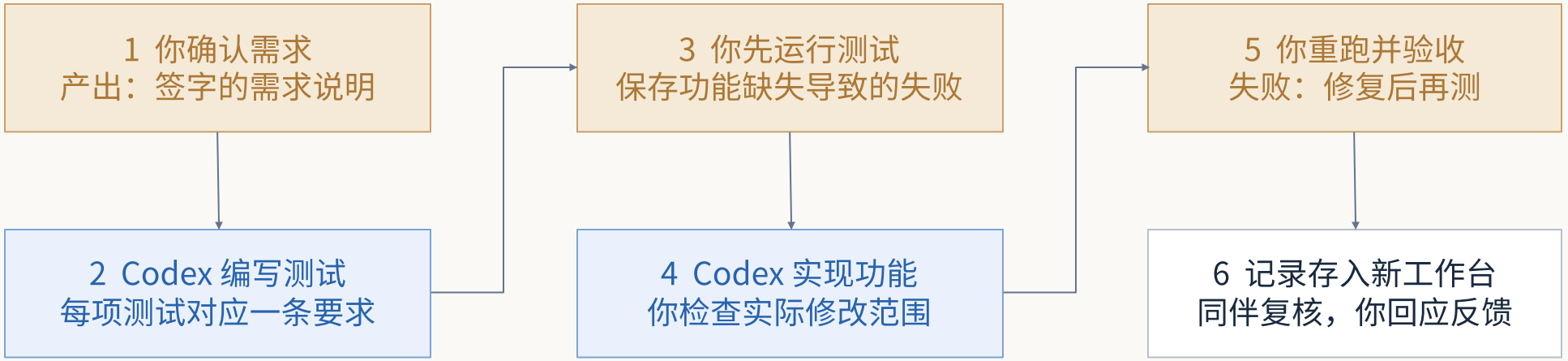
03

做完程序后，怎样让别人也能检查？

以“开发第一版工作台”为例，
先写要求，再修改和测试，
最后根据实际结果决定是否通过。

本讲怎样分工：你作决定，Codex 协助实现

按编号 1—6 阅读。金色表示人的决定和核对，蓝色表示 Codex 的工作。



V0.1 建好前由你安排执行与测试；建好后把开发记录导入。它还不会自动调用 Codex 或运行测试。

一句“帮我做工作台”，还缺哪些决定？

Codex 帮你追问；最终范围由你确认。每个问题都要影响后面的实现或验收。



本例决定：先保存任务和运行记录，关闭后能查回；重复编号拒绝覆盖；网页与自动执行以后再做。

把刚才的决定写成能逐项检查的需求说明

Spec 是本次做什么、做到什么程度的书面约定。下面是 WORKBENCH_SPEC.md 的核心决定。

决定	本讲的具体约定
目标与操作	实现初始化、项目登记、任务创建、记录追加和状态查询。
成功结果	在已登记项目下创建任务 A；退出后重新查询，项目与原需求不变。
错误请求	重复使用任务编号，或向不存在的任务追加记录，都应明确拒绝。
数据保护	错误请求不覆盖旧需求、不删除旧记录，也不留下错误的新记录。
完成与范围	对应测试实际通过，检查修改范围；网页、自动执行和 ERP 功能不做。

每条关键要求都回指：原始痛点 → Codex 建议 → 你的决定。列出允许修改的具体文件，需要扩范围时先重新确认。

代码对照 Diff：改了哪里，为什么要改?

修改前

数据只在程序运行时临时保存在内存里

修改后

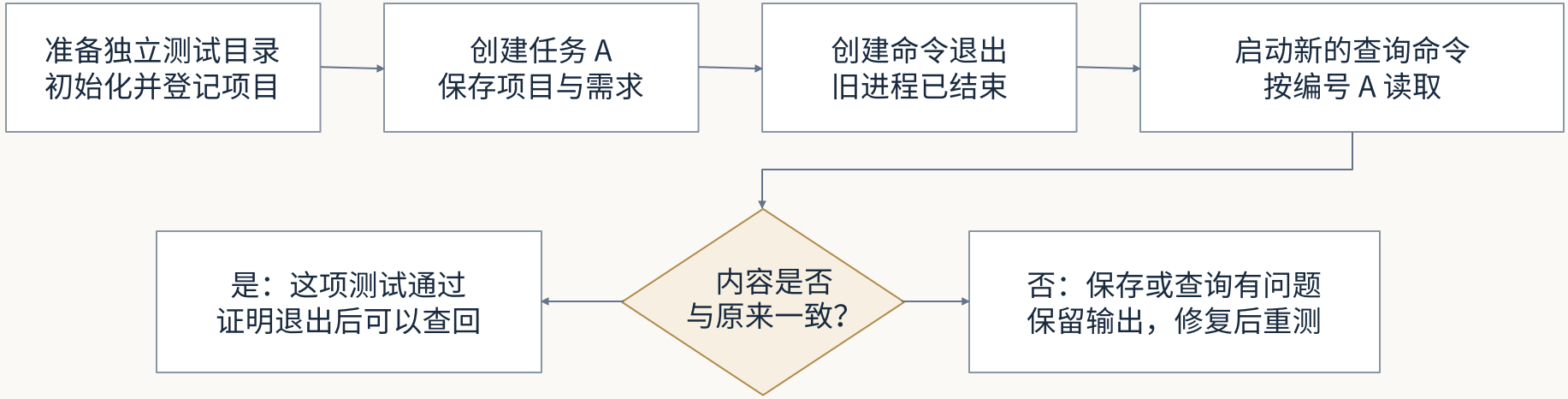
数据写入本地数据库，程序重启后还能读取

打开修改前后的代码对照

检查保存、查询和报错处理是否按要求实现，也要看有没有修改不该动的文件。

这里展示修改思路。加了数据库代码，也可能没有真正保存成功，还需要关闭程序后重新查询。

测试保存功能：关掉程序，再启动查询



测试必须真的创建、退出并重启。只比较内存里的数据，无法验证这项要求。

如果程序一直没有退出，就无法证明关闭后还能查回数据。测试必须经过关闭和重新启动。

同一项测试：先看见失败，再检查修复结果

示意情境：创建任务 A，结束旧进程，再启动新进程查询 A。以下是预期对照，不是实际运行日志。

运行 1：失败

预期：能读回原需求
实际：查询不到任务 A
保留命令、输出、退出码

→

定位并修复

确认缺少保存能力
Codex 补齐保存和读取
你检查修改范围

→

运行 2：通过

重跑同一项测试
实际：项目与原需求一致
保留这一次真实结果

仍失败 → 按输出继续修复再测。通过 → 检查其他验收项，两次记录都保留。

环境或测试出错时先排错；已有功能如实记录。不能伪造失败，也不能用一项测试通过代替全部验收。

测试失败了，保存这条失败记录仍可成功

操作一：运行测试
测试未通过
退出码：非零

→

生成真实运行记录
命令、输出、时间
以及测试退出码

→

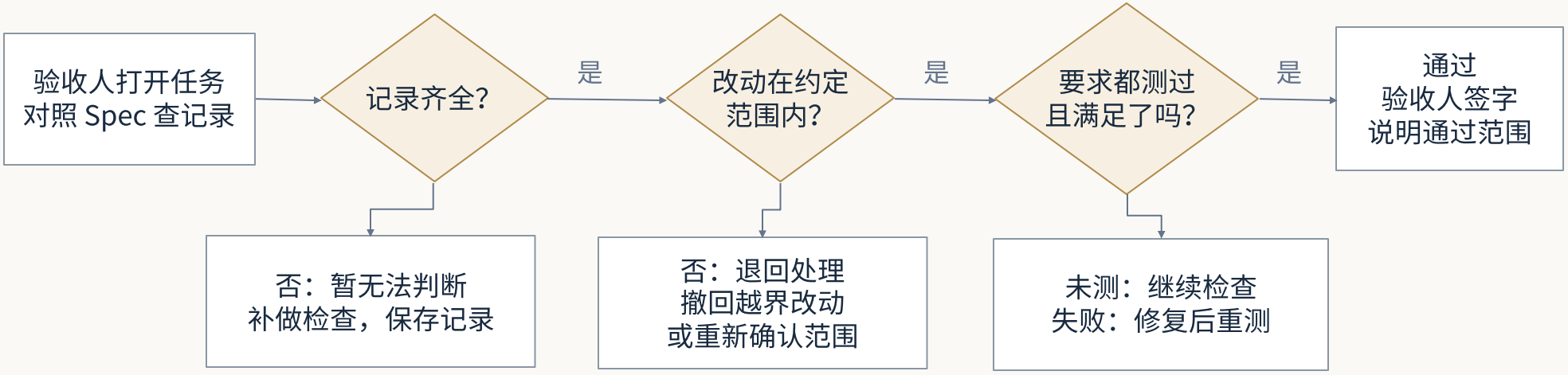
操作二：保存记录
工作台完整保存这条失败
保存操作可以返回成功

保存成功 ≠ 测试通过

以后查询这项任务，仍应看到“测试失败”和原始输出，不能被“保存成功”覆盖。

退出码是命令结束时返回的状态数字，0 通常表示成功。先分清是哪条命令，再结合输出判断结果。

验收时说清：哪些已通过，哪些还没查



退回后补齐记录、处理改动或修复功能，再按这条流程重新验收。

由你根据测试和复核结果决定：通过、退回修改，或暂时无法判断。Codex 说“完成了”不能代替验收。

01 课程最终作品
02 本课要解决的问题
03 从需求到验收
04 工作台 V0.1 设计
05 动手实践
06 总结

04

第一版工作台，应该保存什么？

按一次开发任务的顺序，找出要保存的信息。
项目、需求和运行记录分别保存，再建立关联。

先看使用时要查什么，再决定保存什么

选择项目	项目名称是什么？代码放在哪个目录？新任务属于哪个项目？
开发一项任务	这次要做什么？采用哪份需求？做过哪些修改，运行过哪些检查？
验收或接手旧任务	当时按什么要求检查？哪些测试通过了？谁作了决定，理由是什么？

先回答这些问题，再设计数据和操作。这样才能避免页面做好了，却找不到需要的记录。

一个项目、多项任务，每项任务多次运行

一个项目可以有多项任务，每项任务可以运行多次。



两次运行都保留，并指向同一个任务。不能只在项目上保存“最近一次通过”。

只在项目上写“最近一次通过”，就不知道哪项任务通过了。每个任务和每次运行都要单独保留。

保存旧需求的副本，才能检查旧任务

任务 A 创建时的要求	一周后，新任务提出新要求
重复使用同一编号，必须拒绝	相同内容重复提交，返回原结果
任务 A 按这条要求实现、验收	需求文件被改成了新的规则

只存路径会怎样

需求文件改了，旧任务打开的也是新内容。
于是会拿今天的要求，误判过去的实现。

需求快照就是任务创建时保存的需求副本。它让旧约定还能查到，但不能单靠它证明内容从未被篡改。

准备验收时，先整理这四项内容

工作台提供任务与运行记录；L01 由你整理并核对摘要，不把“记录齐全”当成自动验收。

1 按哪份要求检查？	写明任务编号和当时的需求副本；需求改过要区分版本。
2 实际改了什么？	查看代码对照 Diff，解释各项改动对应哪条要求、是否越界。
3 实际检查了什么？	列出真实命令、原始输出和退出码，注明对应任务与代码版本。
4 还有什么没证明？	列出失败项、漏测项与待确认问题，说明谁来作决定。

例子：旧要求下通过、新要求下失败，应保留并标明两次结果。旧通过不能代替新要求的检查。

拒绝了错误请求，还要确认旧数据没被改

执行什么操作	程序应怎样回复	还要查询确认什么
在已有项目下创建新任务	创建成功	任务属于指定项目
在不存在的项目下创建任务	拒绝创建	没有留下这条任务
用已有编号创建不同任务	拒绝覆盖	原需求和记录没有变化
查询缺少运行记录的任务	说明缺少哪些记录	原有记录仍在

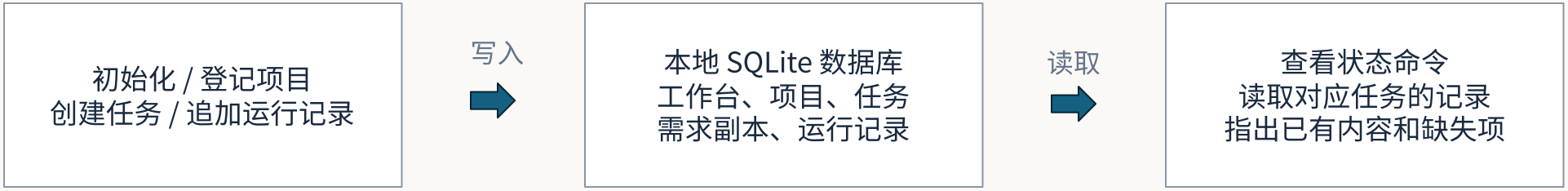
本课选择遇到重复编号就拒绝。其他系统也可以允许更新，但必须先说清什么时候更新、怎样保护旧数据。

第一版要实现五个命令行操作

操作	保存或读取什么	输入的子命令
初始化	谁使用，数据存在哪	workbench-init
登记项目	项目名称与代码目录	workbench-project-add
创建任务	所属项目和当时的需求	workbench-task-create
追加记录	运行命令、输出和结果	workbench-evidence-add
查看状态	已有记录，还缺什么	workbench-status

CLI 就是在终端输入命令来使用程序。用 argparse 读取命令参数，用 SQLite 在本地保存数据。

第一版只负责保存记录和查询记录



状态查询示例（已启用虚拟环境）

```
python -X utf8 -m workbench.cli workbench-status
```

这些命令保存和查询记录。自动调用 Codex、自动运行测试，要在后续课程实现。

程序关闭后数据仍在，叫作持久化。本讲先实现它，网页、自动调用 Codex 和自动测试以后再做。

01 课程最终作品
02 本课要解决的问题
03 从需求到验收
04 工作台 V0.1 设计
05 动手实践
06 总结

05

建好工作台后，把开发记录存进去

按自己确认的需求编写测试、实现和重测。
课程工具先保存输出，新工作台建好后再导入。

开始动手前，先确认起点与分工

先完成 L00 环境准备；在独立练习目录实现，保护课程参考仓库和已有修改。

你先核对

目录：现在在哪个项目、哪个练习目录？
规则：README、AGENTS.md 写了什么？
状态：哪些修改已经存在，不能覆盖？
起点：目标功能和测试是否已经存在？

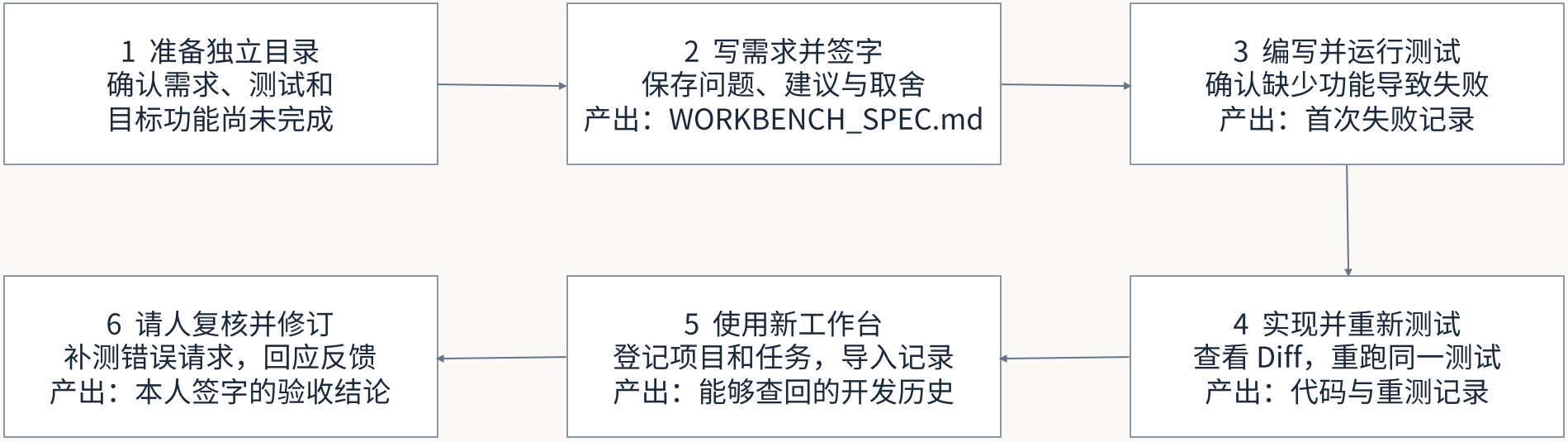
课程已经提供

准备独立练习目录的方法。
evidence.py：保存真实运行输出。
import_evidence.py：导入开发记录。
你和 Codex：形成需求、写测试和五个操作。

在 Git 项目中查看：git rev-parse --show-toplevel ； git status --short
它们说明仓库位置与文件状态，不证明业务测试通过。

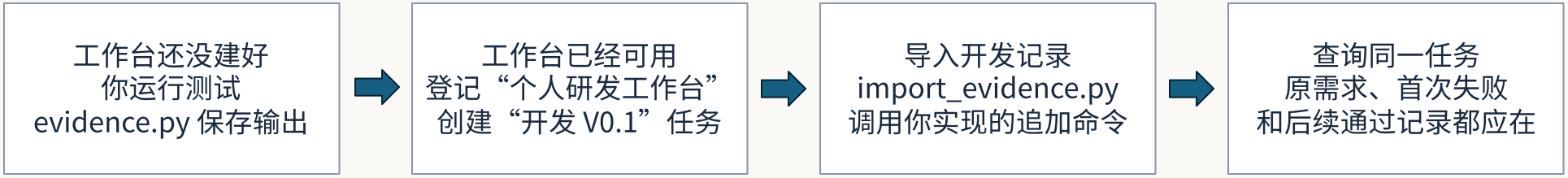
接手检查只是准备。L01 的最终产物仍是五个命令、需求说明、测试记录，以及导入新工作台后可查回的开发历史。

实践步骤：做什么，留下什么记录



每次操作都确认：在哪个目录改代码、按哪份需求开发、用哪条命令测试。完整命令见《实践操作手册》。

自举：用新工作台记录它自己的开发过程



先有开发记录，再有可用工作台，最后把记录补录到自己的任务下。

这次使用验证三件事：能保存、对应正确任务、能查回。失败记录也要原样保留。

第一次使用就能检查：数据存下来了吗？放到正确任务下了吗？还能查出来吗？失败记录也必须保留。

既测正常操作，也测错误请求

实际操作	应该观察到什么
创建任务，退出后重启查询	所属项目和需求内容与创建时一致
用同一编号创建不同任务	请求被拒绝，原需求和记录仍在
给不存在的任务追加记录	请求被拒绝，没有留下这条记录
缺少记录时查询任务状态	指出缺什么，已有记录仍然保留

每项测试都写清：先有什么数据、执行什么操作、应返回什么、原数据应怎样。还要核对测试确实执行了这些步骤。

提交程序，也说明你怎样决定和验证

开发前签字	保存原始问题、Codex 建议和你的取舍。 确认需求说明，并签字。
保存开发过程	提交程序、修改前的测试失败、代码对照、同一测试通过记录， 以及导入工作台的开发记录。
复核后签字	请同伴或教师检查，根据反馈修订。 写明通过、退回或暂时无法判断，并说明理由。

还没收到反馈就写“待复核”。按行动卡检查步骤，用 SUBMISSION.md 提交模板保存材料和两次签字。

01 课程最终作品
02 本课要解决的问题
03 从需求到验收
04 工作台 V0.1 设计
05 动手实践
06 总结

06

换一个项目，你还会设计和检查吗？

先说明为什么要建工作台、怎样分工、
为什么先做记录功能，再试着用于另一个项目。

本讲总结：让一次开发能查找、能复查

为什么需要	把同一任务的需求、修改、测试和验收决定放在一起， 以后还能查找和复查。
怎样协作	你确认需求并作验收决定，Codex 编写测试和实现。 工作台保存每次运行的实际记录。
怎样设计	分别保存项目、任务和运行记录，保留旧需求与失败记录。 用新工作台管理自己的开发任务，检查这些功能是否可用。

本讲完成能保存和查询记录的 V0.1。L04 补齐工作台执行和检查能力后，才首次用它开发 FlowERP。

换一个条件，你还会采用同样的设计吗？

设计解释要落在场景、成本和验证上。先提出自己的选择，再说明用什么结果检验。

条件变化：从一人改成多人

同一任务同时修改，如何避免覆盖？
仍用本地 SQLite 合适吗？
哪些操作需要授权或并发控制？

先列冲突情境，再比较方案与成本。

已有代码：复用还是重写？

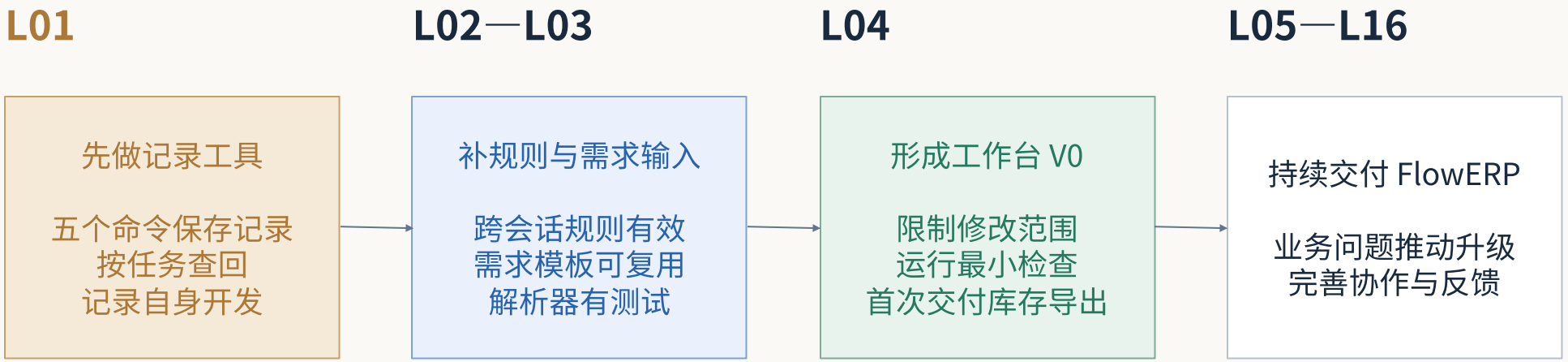
已有保存函数能否保留旧需求？
它的错误处理与本次要求是否一致？
复用会带来哪些依赖和修改影响？

用正常和错误请求检查实际行为。

讨论时请说出一个实际条件、一项取舍、一种代价和一个检查方法。规则与工具名称不能替代这些理由。

从今天的记录工具，走到持续交付工作台

这张路线图说明能力怎样衔接；每一阶段都要用实际结果检查，而不只是增加工具名称。



换成文档校对项目：任务、需求副本和运行记录可以复用；验收标准仍要请编辑重新确认。

下一讲：把仓库规则写进 AGENTS.md

下次和 Codex 对话时，怎样让它继续遵守项目规则？

本讲保存任务和运行记录。
下一讲把长期规则写进 AGENTS.md，
再用实际请求检查 Codex 是否遵守。

课前想一个真实规则，例如“不能覆盖旧任务”。下一讲讨论怎样把它写清楚、检查出来。

课后核对：本讲要交什么，怎样算完成？

形成需求	每项关键要求都说明：遇到了什么问题、Codex 提了什么建议、你最终怎样决定。
实现并重新测试	提交需求说明、实现前失败测试、五个操作、允许范围内的代码修改，以及同一测试通过记录。
实际使用并解释	用自己开发的命令保存本次建设记录。说明本讲先建工作台，L04 才用它开发 FlowERP。

参考仓库通过测试，不代表你已完成。要提交自己练习目录中的代码、真实运行结果和个人判断。

参考：关键词速查

术语	用本讲的例子理解
Spec / Diff	需求说明 / 查看代码修改前后有什么不同
ID / 需求快照	任务的固定编号 / 创建任务时保存的需求副本
红灯 / 绿灯	测试失败 / 测试通过，都要查看实际输出
持久化 / 写集	关掉程序后数据仍在 / 本次允许修改哪些文件
命令证据 / 自举	保存命令和运行结果 / 用新工作台记录自己的开发

术语忘记时查这一页。具体操作见实践手册，需要理解原因和例子时阅读讲义。